

Dry Run Exercises : Solutions

Unit 2 Function Call

Complete the dry run table for each fragment of code given in the questions below. **Give** the value(s) printed to the screen when each is executed.

Follow the module dry run style guide in your answers.

1.

```
public static void add34 ()
{
    int result = add(3, 4);           // L1
    System.out.println("is " + result); // L2

    return;
}

public static int add (int a, int b)           // L3
{
    int res = 0;                               // L4

    System.out.println("Add " + a + " and " + b); // L5
    res = a + b;                               // L6

    return res;                               // L7
}
```

with a test call

```
add34 ();
```

Complete the dry run tables (you may not need all rows):

add34 ()			
LINE	result	add(3, 4) =?	printed
L1	7	add(3, 4) =? 7	Add 3 and 4
L2			is 7

add (3, 4)					
LINE	a	b	res	return res	printed
L3	3	4			
L4			0		
L5					Add 3 and 4
L6			7		
L7				7	

Write EXACTLY what is printed here

Add 3 and 4
is 7

2.

```

public static void add2 ()
{
    int x = 5;           // L1
    int y = 3;           // L2
    int result = add(x, y); // L3
    System.out.println("is " + result); // L4

    return;
}

public static int add (int a, int b)           // L5
{
    int res = 0;                               // L6

    System.out.println("Add " + a + " and " + b); // L7
    res = a + b;                               // L8

    return res;                               // L9
}

```

with a test call

```
add2 ();
```

Complete the dry run tables (you may not need all rows):

add2 ()					
LINE	x	y	result	add(5, 3) =?	printed
L1	5				
L2		3			
L3			8	add(5, 3) =? 8	Add 5 and 3
L4					is 8

add (5, 3)					
LINE	a	b	res	return res	printed
L3	5	3			
L4			0		
L5					Add 5 and 3
L6			8		
L7				8	

Write EXACTLY what is printed here

Add 5 and 3
is 8

3.

```
public static void add2plus ()
{
    int x = 4;           // L1
    int y = 8;           // L2
    int result = addition(x + 3, y + 2); // L3
    System.out.println("Answer: " + result); // L4

    return;
}

public static int addition (int a, int b) // L5
{
    int res = a + b; // L6
    return res; // L7
}
```

with a test call

```
add2plus ();
```

Complete the dry run tables (you may not need all rows):

add2plus ()					
LINE	x	y	result	addition(x+3, y+2) =?	printed
L1	4				
L2		8			
L3			17	addition(7,10) =? 17	
L4					Answer: 17

addition (7, 10)				
LINE	a	b	res	return res
L5	7	10		
L6			17	
L7				17

Write EXACTLY what is printed here

Answer: 17

```

4.
public static int double1 (int number_to_double)           // L1
{
    int doubled = number_to_double * 2;                   // L2

    return doubled;                                         // L3
} // END double1

public static void doubleOranges ()
{
    final int N = 7;                                       // L4

    int oranges = double1(N);                             // L5
    System.out.println(oranges);                          // L6

    return;                                                // L7
} // END doubleOranges

```

with a test call

```
doubleOranges ();
```

Complete the dry run tables (you may not need all rows):

doubleOranges ()					
LINE	N	oranges	double1(N)	return	printed
L4	7				
L5		14	double1(7)=? 14		
L6					14
L7				void	

double1 (7)			
LINE	number_to_double	doubled	return doubled
L1	7		
L2		14	
L3			14

Write EXACTLY what is printed here

14

```

5.
public static int double2 (int number_to_double)           // L1
{
    return (number_to_double * 2);                         // L2
} // END double2

public static void doublePears ()
{
    final int N_PEARs = 4;                                 // L3

    int pears = double2(N_PEARs);                          // L4
    System.out.println(pears);                             // L5

    return;                                                // L6
} // END doublePears

```

with a test call

```
doublePears ();
```

Complete the dry run tables (you may not need all rows):

doublePears ()					
LINE	N_PEARs	pears	double2(N_PEARs)	return	printed
L3	4				
L4		8	double2(4)=?8		
L5					8
L6				void	

double2 (4)		
LINE	number_to_double	return (number_to_double * 2)
L1	4	
L2		8

Write EXACTLY what is printed here

8

6.

```
public static int double3 (int number_to_double)           // L1
{
    return (number_to_double * 2);                         // L2
} // END double3
```

```
public static int triple3 (int number_to_triple)           // L3
{
    int result = number_to_triple * 3;                     // L4
    return result;                                         // L5
} // END triple3
```

```
public static void x6Bananas ()
{
    final int N_BANANAS = 11;                             // L6

    int bananas = double3(N_BANANAS);                      // L7
    bananas = triple3(bananas);                             // L8

    System.out.println(bananas);                           // L9

    return;                                                 // L10
} // END x6Bananas
```

with a test call

```
x6Bananas ()
```

Complete the dry run tables (you may not need all rows):

x6Bananas ()						
LINE	N_BANANAS	bananas	double3(N-BANANAS)	triple3(bananas)	return	printed
L6	11					
L7		22	double3(11)=? 22			
L8		66		triple3(22)=?66		
L9						66
L10					void	

triple3 (22)			
LINE	number_to_triple	result	return result
L3	22		
L4		66	
L5			66

double3 (11)		
LINE	number_to_double	return (number_to_double * 2)
L1	11	
L2		22

Write EXACTLY what is printed here

66


```

7.
public static int double4 (int number_to_double)           // L1
{
    return (number_to_double * 2);                         // L2
} // END double4

public static int triple4 (int number_to_triple)           // L3
{
    return (number_to_triple * 3);                         // L4
} // END triple4

public static void x6Bananas2 ()
{
    final int N_BANANAS = 11;                             // L5

    int bananas = triple4(N_BANANAS);                      // L6
    bananas = double4(bananas);                            // L7

    System.out.println(bananas);                           // L8

    return;                                                // L9
} // END x6Bananas2

```

with a test call

```
x6Bananas2 ()
```

Complete the dry run tables (you may not need all rows):

x6Bananas2 ()						
LINE	N_BANANAS	bananas	triple4(N-BANANAS)	double4(bananas)	return	printed
L5	11					
L6		33	triple4(11)=?33			
L7		66		double4(33)=? 66		
L8						66
L9					void	

triple4 (11)		
LINE	number_to_triple	return (number_to_triple * 3)
L3	11	
L4		33

double4 (33)		
LINE	number_to_double	return (number_to_double * 2)
L1	33	
L2		66

Write EXACTLY what is printed here

66

8.

```

public static int double5 (int number_to_double)           // L1
{
    return (number_to_double * 2);                         // L2
} // END double5

public static void quadruplePears ()
{
    final int N_PEARS = 4;                                 // L3

    int pears = double5(N_PEARS);                           // L4
    pears = double5(pears);                                 // L5

    System.out.println(pears);                             // L6

    return;                                                 // L7
} // END quadruplePears
with a test call
    quadruplePears ()

```

Complete the dry run tables (you may not need all rows):

quadruplePears ()						
LINE	N_PEARS	pears	double5(N-PEARS)	double5(pears)	return	printed
L3	4					
L4		8	double5(4)=? 8			
L5		16		double5(8)=? 16		
L6						16
L7					void	

double5 (4)		
LINE	number_to_double	return (number_to_double * 2)
L1	4	
L2		8

double5 (8)		
LINE	number_to_double	return (number_to_double * 2)
L1	8	
L2		16

Write EXACTLY what is printed here

16

9.

```
public static int double6 (int number_to_double)           // L1
{
    return (number_to_double * 2);                         // L2
} // END double6
```

```
public static void quadruplePears2 ()
{
    final int N_PEARS = 11;                                // L3

    int pears = double6(double6(N_PEARS));                 // L4

    System.out.println(pears);                             // L5

    return;                                                 // L6
} // END quadruplePears2
```

with a test call

```
quadruplePears2 ()
```

Complete the dry run tables (you may not need all rows):

quadruplePears2 ()						
LINE	N_PEARS	pears	double6(N-PEARS)	double6(double6(...))	return	printed
L3	11					
L4		44	double6(11)=? 22	double6(22)=? 44		
L5						44
L6					void	

double6 (11)		
LINE	number_to_double	return (number_to_double * 2)
L1	11	
L2		22

double6 (2)		
LINE	number_to_double	return (number_to_double * 2)
L1	22	
L2		44

Write EXACTLY what is printed here

44

10.

```
public static int square (int to_square)           // L1
{
    int squared = to_square * to_square;          // L2

    System.out.print(squared);                     // L3

    return squared;                                // L4
} // END square
```

```
public static void squaresquaredCherries ()
{
    final int N_CHERRIES = 3;                      // L5

    int cherries = square(square(N_CHERRIES));      // L6

    System.out.println(cherries);                  // L7

    return;                                         // L8
} // END squaresquaredCherries
```

with a test call

```
squaresquaredCherries ()
```

Complete the dry run tables (you may not need all rows):

squaresquaredCherries ()						
LINE	N_CHERRIES	cherries	square(N-CHERRIES)	square(square(...))	return	printed
L5	3					
L6		81	square(3)=? 9	square(9)=? 81		981
L7						81
L8					void	

square (3)				
LINE	to_square	squared	return (squared)	printed
L1	3			
L2		9		
L3				9
L4			9	

square (9)				
LINE	to_square	squared	return (squared)	printed
L1	9			
L2		81		
L3				81
L4			81	

Write EXACTLY what is printed here

98181

